

INNOPOLIS UNIVERSITY

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 10 · Гомография и масштабирование изображения

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

«Introduction to Computer Vision» course, Innopolis University (IU), Russia, Fall 2026

Карта курса

Блок I Изображение и его источник	**Блок II** Операции над изображением	**Блок III** Признаки	**Блок IV** Гомография и геометрия	**Блок V** Видео и движение
L1 Введение. Изображение и видео как данные	**L4** Геометрические преобразования	**L7** Границы, линии и робастное оценивание	**L10 Гомография и масштабирование изображения**	**L13** Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	**L5** Пространственная фильтрация	**L8** Точки интереса: детекторы	**L11** Модель камеры и калибровка	**L14** Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	**L6** Частотная область и пирамиды	**L9** Дескрипторы и сопоставление	**L12** Стерео и эпиполярная геометрия	**L15** Синтез: от двух видов к 3D. Итоги

Сегодня — L10: Гомография: оценивание и выравнивание

1 Гомография: оценивание и выравнивание (RECAP)

Термины: гомография (проективное преобразование), аффинное преобразование, матрица трансформации · поиск соответствий

Что такое гомография?

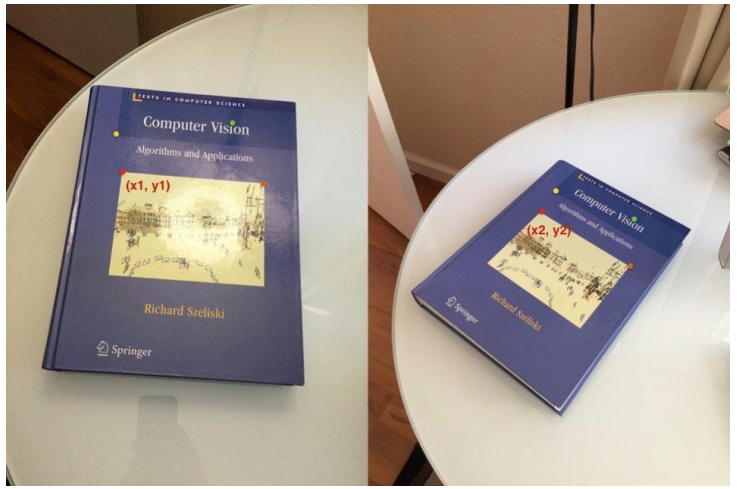
- Рассмотрим два изображения плоскости (верхняя часть книги). Красные, зеленые, желтые и оранжевые точки представляют одну и ту же физическую точку на двух изображениях (т.е. соответствующие точки).
- Гомография — это преобразование (матрица 3×3), которое отображает точки на одном изображении в соответствующие точки на другом изображении.

$$H = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix}$$

- Рассмотрим набор соответствующих красных точек: (x1, y1) на первом изображении и (x2, y2) на втором изображении. Тогда гомография H отображает их следующим образом:

$$\begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = H \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix}$$

- Вышеуказанное уравнение верно для ВСЕХ наборов соответствующих точек, если они лежат на одной плоскости в реальном мире. Другими словами, вы можете применить гомографию к левому изображению, и книга на левом изображении будет выровнена с книгой на правом изображении.



If apply homography to the left image

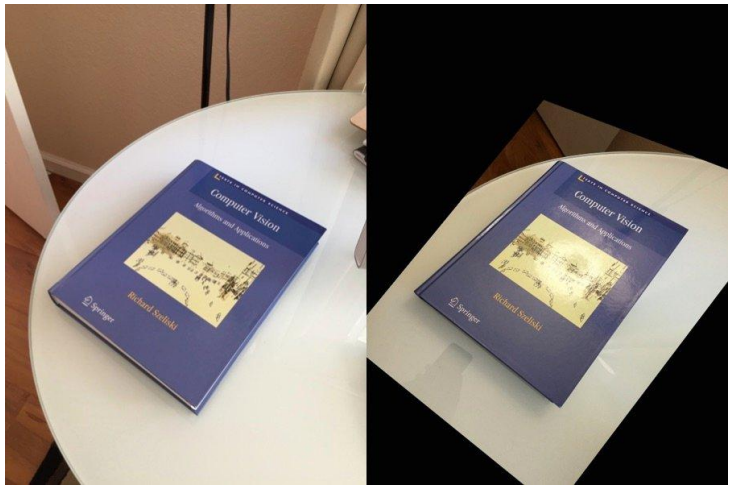


Fig. © Satya Mallick, et al. Homography examples using OpenCV (Python/C ++), LearnOpenCV, 2021, https://learnopencv.com/homography-examples-using-opencv-python-c/

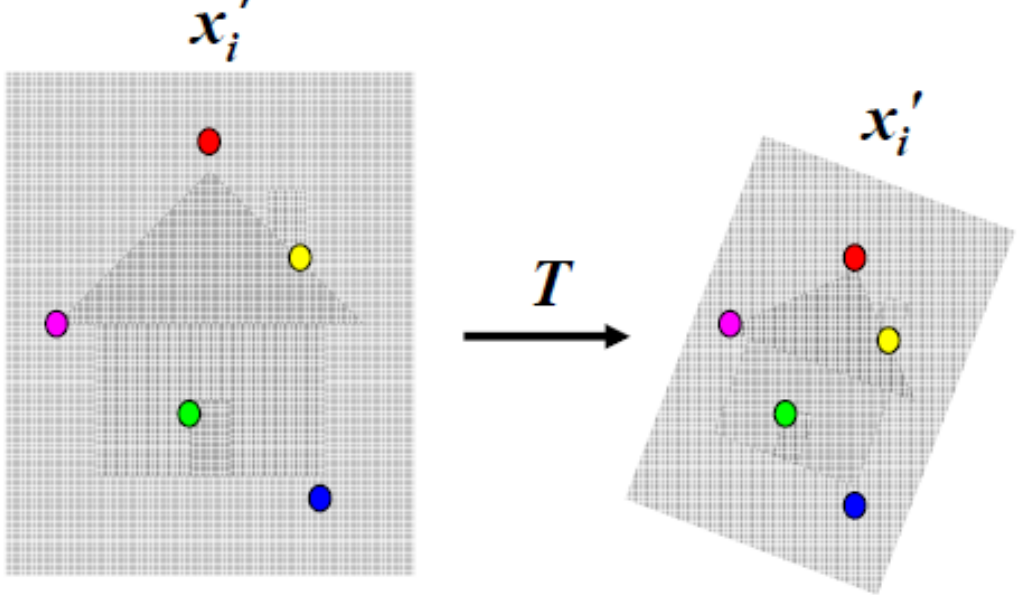
Аффинное преобразование: три пары точек

1. **Математика**: Правило 3 точек (6 уравнений). В формулах аффинного сдвига есть 6 неизвестных коэффициентов (a, b, c, d, tx, ty). Каждая точка дает две координаты (x) и (y), то есть 2 уравнения. Чтобы однозначно найти всю матрицу в OpenCV с помощью функции cv2.getAffineTransform, строго необходимо и достаточно знать координаты всего 3 точек до и после изменения.
2. **Геометрия**: Что происходит с фигурами. Аффинное преобразование — это «всё линейное»: оно может крутить объект, менять масштаб отдельно по вертикали и горизонтали, а также сдвигать слоями (shear). Главный инвариант: Параллельные линии всегда остаются параллельными (квадрат превращается в параллелограмм, но не в трапецию; круг становится эллипсом).
3. **Определитель матрицы гомографии** $\det A$ **и площади**: Число $\det A$ (детерминант линейной части) показывает, во сколько раз изменится площадь объекта. На примере: площадь квадрата уменьшилась на 7% (умножилась на 0.93). Если $\det \mathrm{A} < 0$, это значит, что картинку не просто повернули, а отзеркалили.
4. **Ограничение метода:** аффинного преобразования недостаточно для выпрямления плакатов, снятых под углом, потому что оно принципиально не умеет сводить параллельные линии в точку схода (для этого нужна гомография с её 8 параметрами и 4 точками).

Аффинное — «всё линейное»: скос и разномасштабность есть, но параллельные линии не сходятся. Для плаката под углом его мало

Проблема поиска соответствий

- In alignment, we will fit the parameters of some transformation according to a set of matching feature pairs ("correspondences").

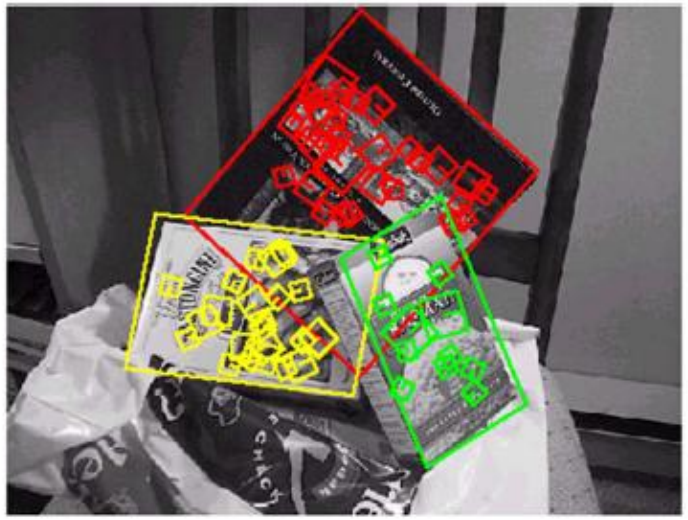


© Slide credit: Kristen Grauman

Трансформации в анализе паттернов и форм

- Simple fitting procedure (linear least squares)
- Approximates viewpoint changes for roughly planar objects and roughly orthographic cameras
- Can be used to initialize fitting for more complex models

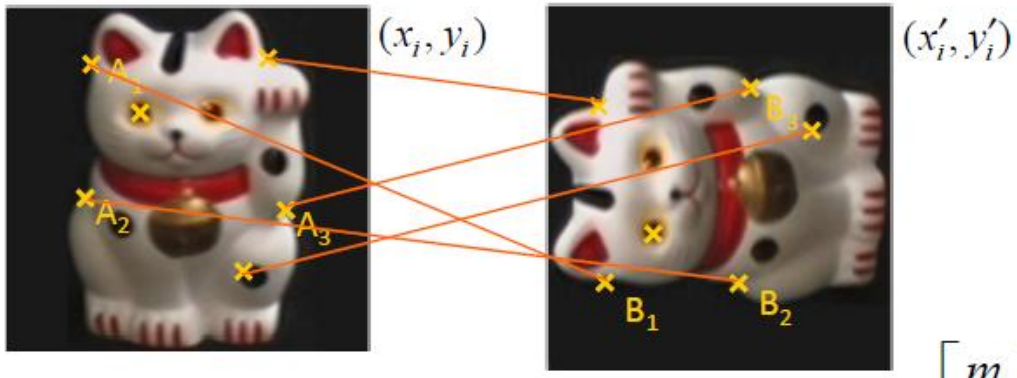




© Slide credit: Svetlana Lazebnik, David Lowe

Трансформации в анализе паттернов и форм

- Assuming we know the correspondences, how do we get the transformation?



$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} \Rightarrow \begin{bmatrix} \quad \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} \quad \end{bmatrix}$$

© Slide credit: Bastian Leibe

Трансформации в анализе паттернов и форм

- Assuming we know the correspondences, how do we get the transformation?

$$\begin{bmatrix} x_i' \\ y_i' \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} \Rightarrow \begin{bmatrix} & \cdots & & & & \\ x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ & \cdots & & & & \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} \cdots \\ x_i' \\ y_i' \\ \cdots \end{bmatrix}$$

- How many matches (correspondence pairs) do we need to solve for the transformation parameters?
- Once we have solved for the parameters, how do we compute the coordinates of the corresponding point for (x_{new}, y_{new})?

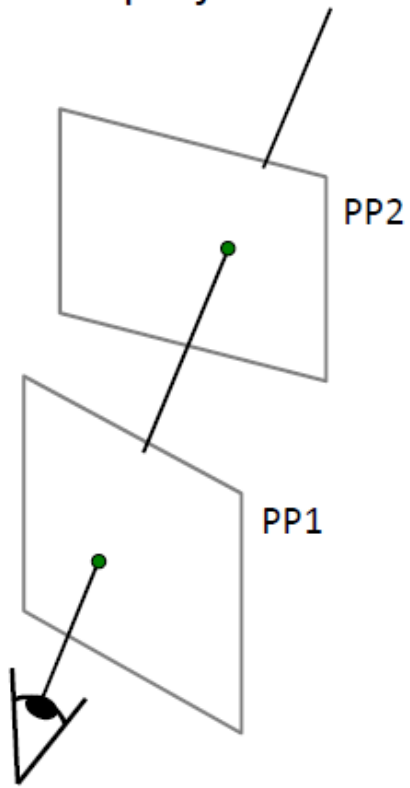
© Slide credit: Bastian Leibe

Подгонка проективного преобразования

- A projective transform is a mapping between any two perspective projections with the same center of projection.
  - I.e. two planes in 3D along the same sight ray
- Properties
  - Rectangle should map to arbitrary quadrilateral
  - Parallel lines aren't
  - but must preserve straight lines
- This is called a homography

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

p' H p



© Slide credit: Alyosha Efros

Подгонка проективного преобразования

- A projective transform is a mapping between any two perspective projections with the same center of projection.
  - I.e. two planes in 3D along the same sight ray
- Properties
  - Rectangle should map to arbitrary quadrilateral
  - Parallel lines aren't
  - but must preserve straight lines
- This is called a homography

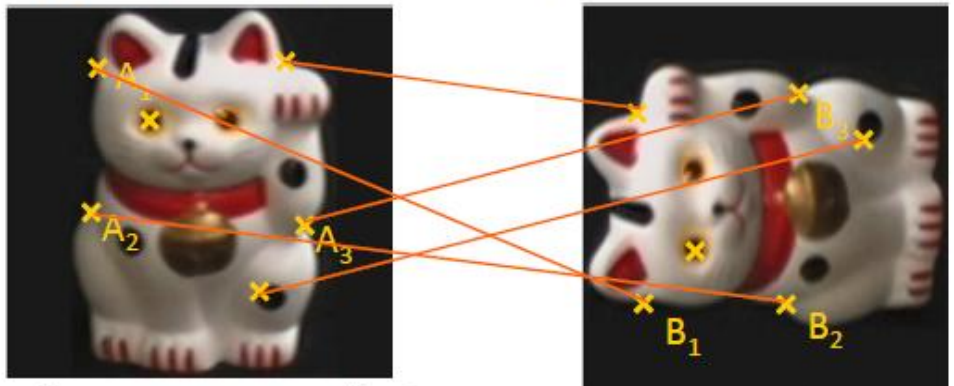
$$\underbrace{\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix}}_{p'} = \underbrace{\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}}_{H} \underbrace{\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}}_{p}$$

Set scale factor to 1
⇒ 8 parameters left.

© Slide credit: Alyosha Efros

Проективное преобразование (Гомография)

- Estimating the transformation



Homogenous coordinates Image coordinates

$$\mathbf{X}_{A_1} \leftrightarrow \mathbf{X}_{B_1}$$
$$\mathbf{X}_{A_2} \leftrightarrow \mathbf{X}_{B_2}$$
$$\mathbf{X}_{A_3} \leftrightarrow \mathbf{X}_{B_3}$$
$$\vdots$$

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x'' \\ y'' \\ 1 \end{bmatrix} = \frac{1}{z'} \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$

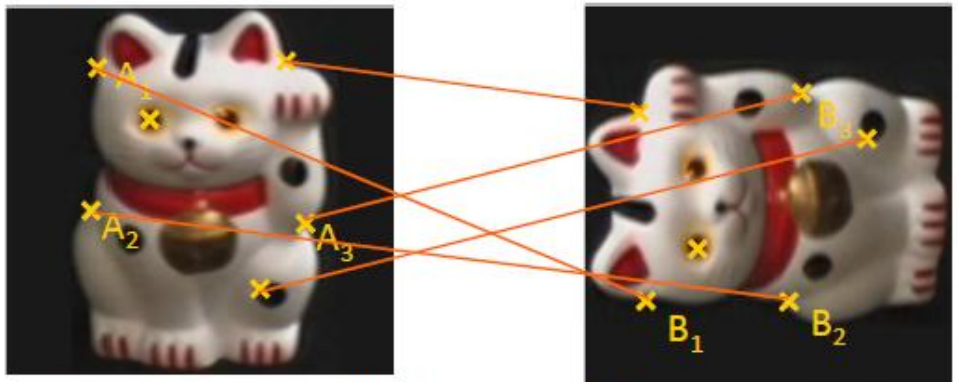
Matrix notation

$$x' = Hx$$
$$x'' = \frac{1}{z'} x'$$

© Slide credit: Krystian Mikolajczyk

Проективное преобразование (Гомография)

- Estimating the transformation



Homogenous coordinates Image coordinates

$$\mathbf{X}_{A_1} \leftrightarrow \mathbf{X}_{B_1}$$
$$\mathbf{X}_{A_2} \leftrightarrow \mathbf{X}_{B_2}$$
$$\mathbf{X}_{A_3} \leftrightarrow \mathbf{X}_{B_3}$$
$$\vdots$$

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x'' \\ y'' \\ 1 \end{bmatrix} = \frac{1}{z'} \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$

$$x_{A_1} = \frac{h_{11}\, x_{B_1} + h_{12} y_{B_1} + h_{13}}{h_{31}\, x_{B_1} + h_{32} y_{B_1} + 1}$$

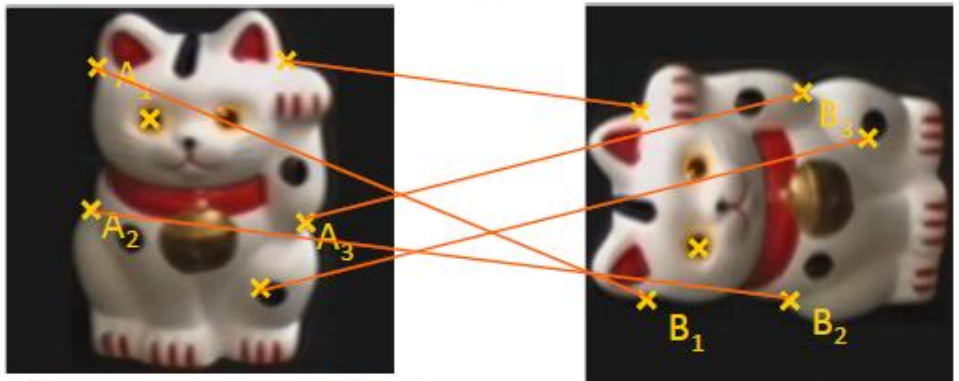
Matrix notation

$$x' = Hx$$
$$x'' = \frac{1}{z'} x'$$

© Slide credit: Krystian Mikolajczyk

Проективное преобразование (Гомография)

- Estimating the transformation



Homogenous coordinates Image coordinates

$$\mathbf{X}_{A_1} \leftrightarrow \mathbf{X}_{B_1}$$
$$\mathbf{X}_{A_2} \leftrightarrow \mathbf{X}_{B_2}$$
$$\mathbf{X}_{A_3} \leftrightarrow \mathbf{X}_{B_3}$$
$$\vdots$$

$$\begin{bmatrix} x' \\ y' \\ z' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x'' \\ y'' \\ 1 \end{bmatrix} = \frac{1}{z'} \begin{bmatrix} x' \\ y' \\ z' \end{bmatrix}$$

$$x_{A_1} = \frac{h_{11}\, x_{B_1} + h_{12} y_{B_1} + h_{13}}{h_{31}\, x_{B_1} + h_{32} y_{B_1} + 1} \qquad y_{A_1} = \frac{h_{21}\, x_{B_1} + h_{22} y_{B_1} + h_{23}}{h_{31}\, x_{B_1} + h_{32} y_{B_1} + 1}$$

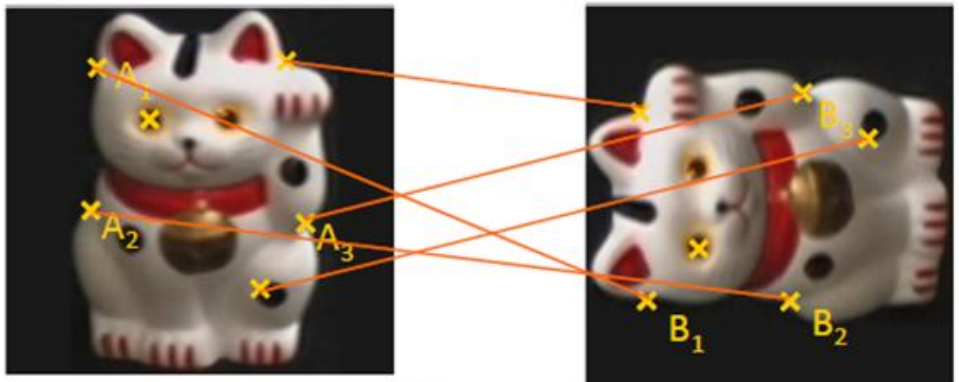
Matrix notation

$$x' = Hx$$
$$x'' = \frac{1}{z'} x'$$

© Slide credit: Krystian Mikolajczyk

Проективное преобразование (Гомография)

- Estimating the transformation



Homogenous coordinates Image coordinates

$$\mathbf{X}_{A_1} \leftrightarrow \mathbf{X}_{B_1}$$
$$\mathbf{X}_{A_2} \leftrightarrow \mathbf{X}_{B_2}$$
$$\mathbf{X}_{A_3} \leftrightarrow \mathbf{X}_{B_3}$$
$$\vdots$$

$$x_{A_1} = \frac{h_{11}\,x_{B_1} + h_{12}\,y_{B_1} + h_{13}}{h_{31}\,x_{B_1} + h_{32}\,y_{B_1} + 1} \qquad y_{A_1} = \frac{h_{21}\,x_{B_1} + h_{22}\,y_{B_1} + h_{23}}{h_{31}\,x_{B_1} + h_{32}\,y_{B_1} + 1}$$

$$x_{A_1} h_{31}\,x_{B_1} + x_{A_1} h_{32}\,y_{B_1} + x_{A_1} = h_{11}\,x_{B_1} + h_{12}\,y_{B_1} + h_{13}$$

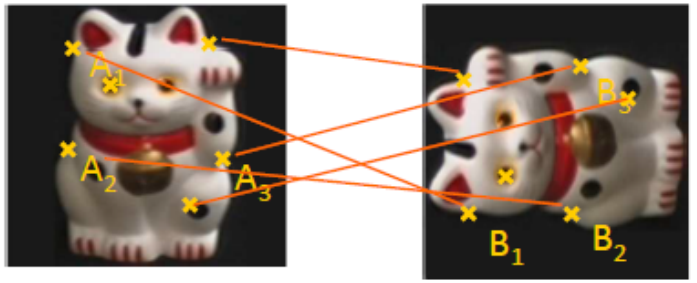
$$h_{11}\,x_{B_1} + h_{12}\,y_{B_1} + h_{13} - x_{A_1} h_{31}\,x_{B_1} - x_{A_1} h_{32}\,y_{B_1} - x_{A_1} = 0$$
$$h_{21}\,x_{B_1} + h_{22}\,y_{B_1} + h_{23} - y_{A_1} h_{31}\,x_{B_1} - y_{A_1} h_{32}\,y_{B_1} - y_{A_1} = 0$$

© Slide credit: Krystian Mikolajczyk

Проективное преобразование (Гомография)

- Estimating the transformation

$$h_{11}\,x_{B_1} + h_{12}\,y_{B_1} + h_{13} - x_{A_1}\,h_{31}\,x_{B_1} - x_{A_1}\,h_{32}\,y_{B_1} - x_{A_1} = 0$$
$$h_{21}\,x_{B_1} + h_{22}\,y_{B_1} + h_{23} - y_{A_1}\,h_{31}\,x_{B_1} - y_{A_1}\,h_{32}\,y_{B_1} - y_{A_1} = 0$$



$$\mathbf{X}_{A_1} \leftrightarrow \mathbf{X}_{B_1}$$
$$\mathbf{X}_{A_2} \leftrightarrow \mathbf{X}_{B_2}$$
$$\mathbf{X}_{A_3} \leftrightarrow \mathbf{X}_{B_3}$$
$$\vdots$$

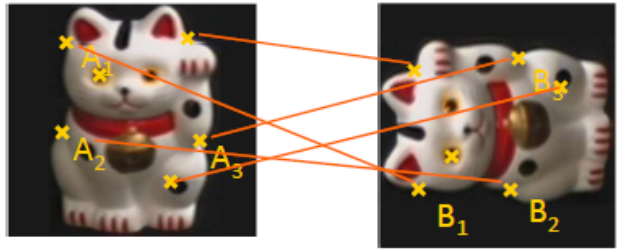
$$\begin{bmatrix} x_{B_1} & y_{B_1} & 1 & 0 & 0 & 0 & -x_{A_1}x_{B_1} & -x_{A_1}y_{B_1} & -x_{A_1} \\ 0 & 0 & 0 & x_{B_1} & y_{B_1} & 1 & -y_{A_1}x_{B_1} & -y_{A_1}y_{B_1} & -y_{A_1} \\ . & . & . & . & . & . & . & . & . \\ . & . & . & . & . & . & . & . & . \\ . & . & . & . & . & . & . & . & . \end{bmatrix} \cdot \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ . \\ . \\ . \end{bmatrix}$$

$$Ah = 0$$

© Slide credit: Krystian Mikolajczyk

Проективное преобразование (Гомография)

- Estimating the transformation
- Solution:
  - Null-space vector of A
  - Corresponds to smallest eigenvector



$$Ah = 0$$

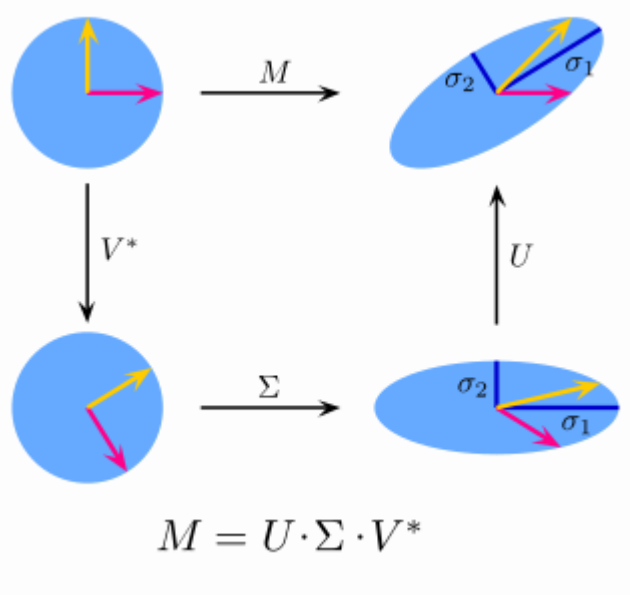
SVD

$$\mathbf{X}_{A_1} \leftrightarrow \mathbf{X}_{B_1}$$
$$\mathbf{X}_{A_2} \leftrightarrow \mathbf{X}_{B_2}$$
$$\mathbf{X}_{A_3} \leftrightarrow \mathbf{X}_{B_3}$$
$$\vdots$$

$$\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{V}^T = \mathbf{U}\begin{bmatrix} d_{11} & \cdots & d_{19} \\ \vdots & \ddots & \vdots \\ d_{91} & \cdots & d_{99} \end{bmatrix}\begin{bmatrix} v_{11} & \cdots & v_{19} \\ \vdots & \ddots & \vdots \\ v_{91} & \cdots & v_{99} \end{bmatrix}^T$$

$$\mathbf{h} = \frac{[v_{19}, \cdots, v_{99}]}{v_{99}}$$

Minimizes least square error



$$M = U \cdot \Sigma \cdot V^*$$

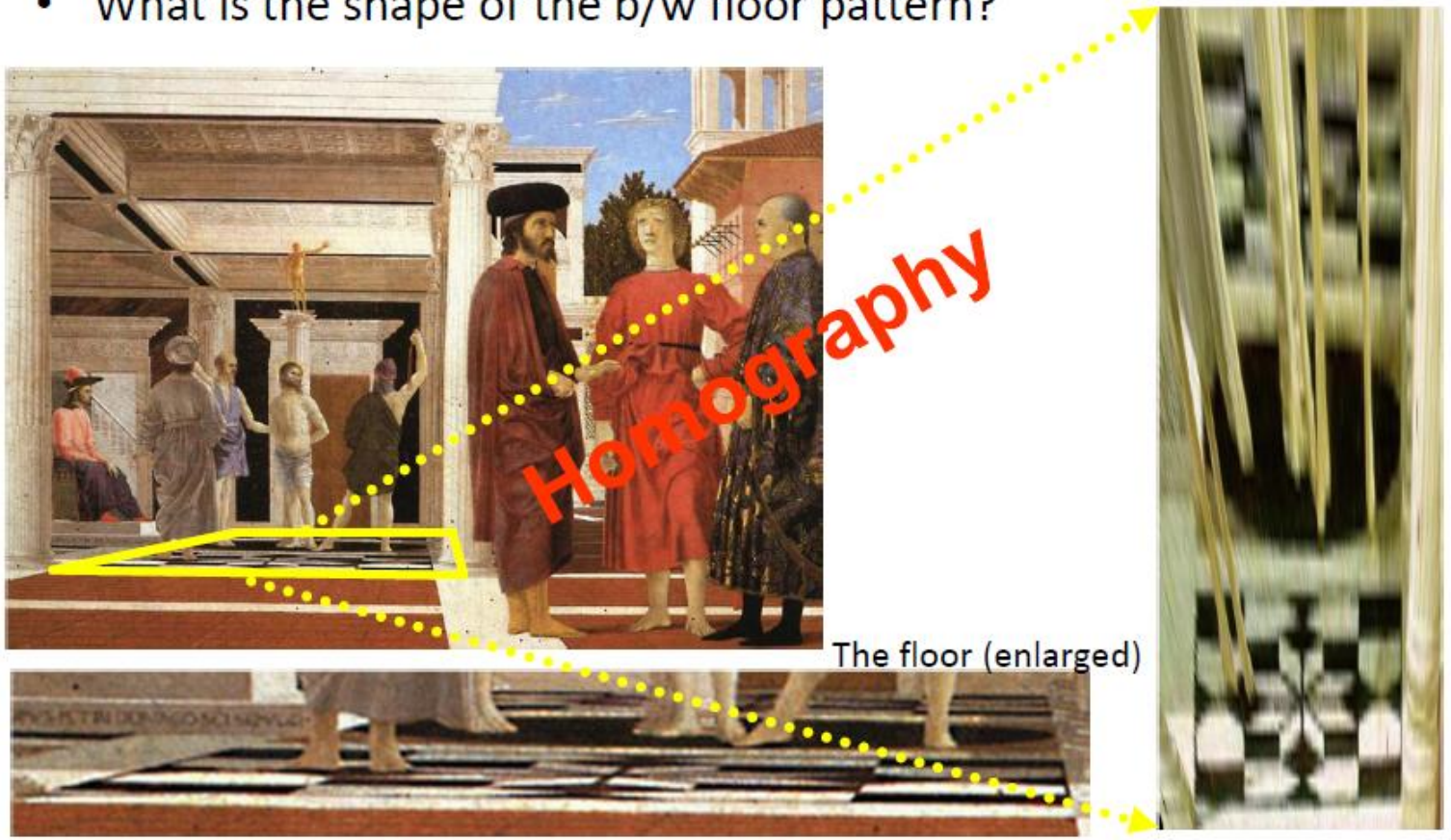
Если конфигурация точек хорошая (общее положение, 4 и более не коллинеарных точек), минимальное число σ_9 будет строго изолированным, и вектор *h* будет единственным (с точностью до знака). Если минимальное сингулярное число повторяется (например, $\sigma_8 = \sigma_9 = 0$). То столбцы v_8, v_9 можно вращать внутри этого подпространства как угодно, порождая бесконечно много разных векторов *h*. Но это случается строго в вырожденных случаях: когда точек меньше 4, или 3 из 4 точек лежат на одной прямой.

© Slide credit: Krystian Mikolajczyk

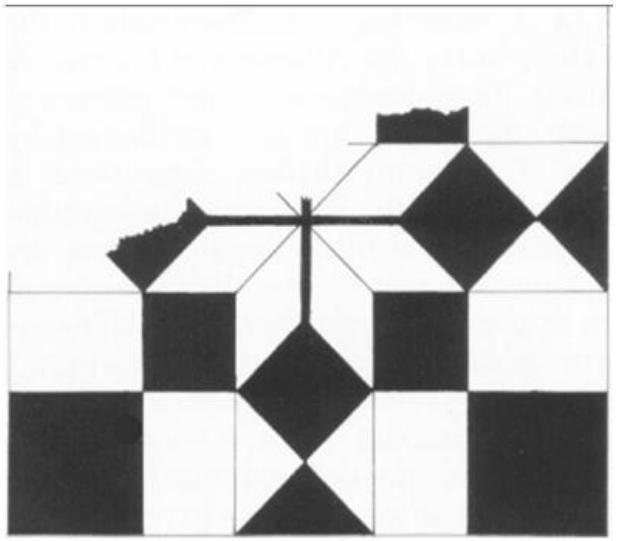
© Wiki: Singular value decomposition

Трансформации в анализе паттернов и форм

- What is the shape of the b/w floor pattern?



vs



From Martin Kemp *The Science of Art*
(manual reconstruction)

© Slide credit: Fei-Fei Li

Как вычислить гомографию?

Для вычисления гомографии между двумя изображениями необходимо знать как минимум четыре пары соответствующих точек. Если таких пар больше четырех — это даже лучше: OpenCV надежно вычислит гомографию, наилучшим образом соответствующую всем этим точкам.

```python
#!/usr/bin/env python

import cv2
import numpy as np

if __name__ == '__main__' :

    # Read source image.
    im_src = cv2.imread('book2.jpg')
    # Four corners of the book in source image
    pts_src = np.array([[141, 131], [480, 159], [493, 630],[64, 601]])

    # Read destination image.
    im_dst = cv2.imread('book1.jpg')
    # Four corners of the book in destination image.
    pts_dst = np.array([[318, 256],[534, 372],[316, 670],[73, 473]])

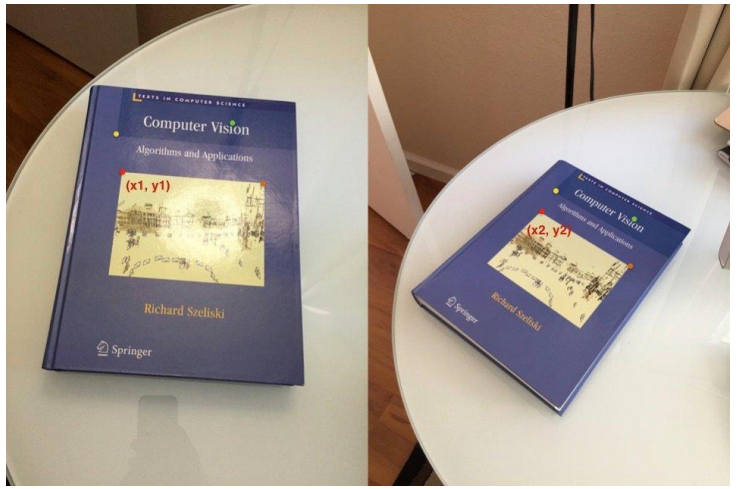
    # Calculate Homography
    h, status = cv2.findHomography(pts_src, pts_dst)

    # Warp source image to destination based on homography
    im_out = cv2.warpPerspective(im_src, h, (im_dst.shape[1],im_dst.shape[0]))

    # Display images
    cv2.imshow("Source Image", im_src)
    cv2.imshow("Destination Image", im_dst)
    cv2.imshow("Warped Source Image", im_out)

    cv2.waitKey(0)
```

Обычно соответствия между точками находятся автоматически путем сопоставления таких признаков, как SIFT или SURF, на разных изображениях, но в этой статье мы просто будем отмечать точки вручную.



Применяя гомографию к левому изображению

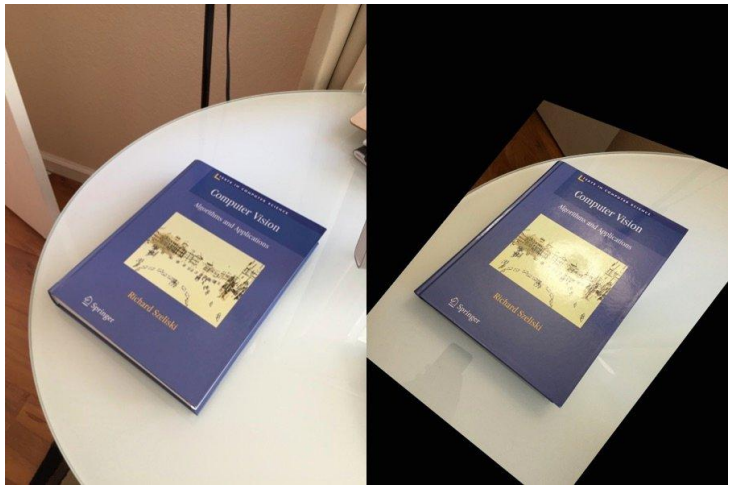


Fig. © Satya Mallick, et al. Homography examples using OpenCV (Python/C ++), LearnOpenCV, 2021, https://learnopencv.com/homography-examples-using-opencv-python-c/

Свойства и цена

Аффинное (6 DoF)

- три пары точек, система всегда решается, если точки не на прямой
- параллельность сохранена — удобно для сеток и текста
- стоит `warpAffine` ≈ 3 мс; описывает поворот, масштаб, скос
- **не описывает** плакат под углом: дальний край не сжимается

Проективное (8 DoF)

- четыре пары точек; работает только для плоскости или чистого поворота камеры
- параллельность потеряна, прямые сохранены; растяжение неравномерно
- стоит `warpPerspective` ≈ 3.5 мс: деление на w в каждом пикселе
- **ломается** на дисторсии, на не-плоскости, на горизонте в кадре

2

Камера, функция рассеяния точки и что с ними делает resize

Термины: PSF (Point Spread Function), полоса B, дискретное ядро · идея вторичных источников (пиксели Гюйгенса)

Чему научимся сегодня

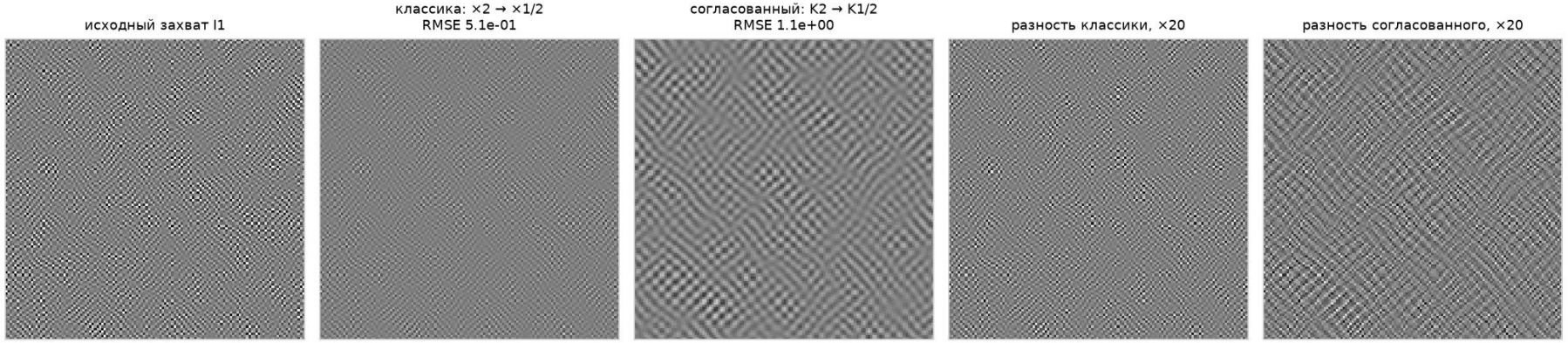
LEARNING OBJECTIVES

- Объяснить PSF и формулу формирования кадра `I[n] = (Icont * h)(n)`: что такое h и откуда полоса B
- Разложить любой resize на четыре фазы и предсказать, чем цепочка ресайзов отличается от одного шага
- Расшифровать мастер-формулу `Ks = K0 Ms K∞` и её концы: `K0`, тождество, `K∞`
- Проверить кодом три несогласованности практики: цикл, композицию, коммутативность
- Выбирать: согласованное ядро или обычный AA — по задаче (SR, сопоставление, аугментации)

Цикл, цепочка, порядок: библиотеки ошибаются

МОТИВАЦИЯ

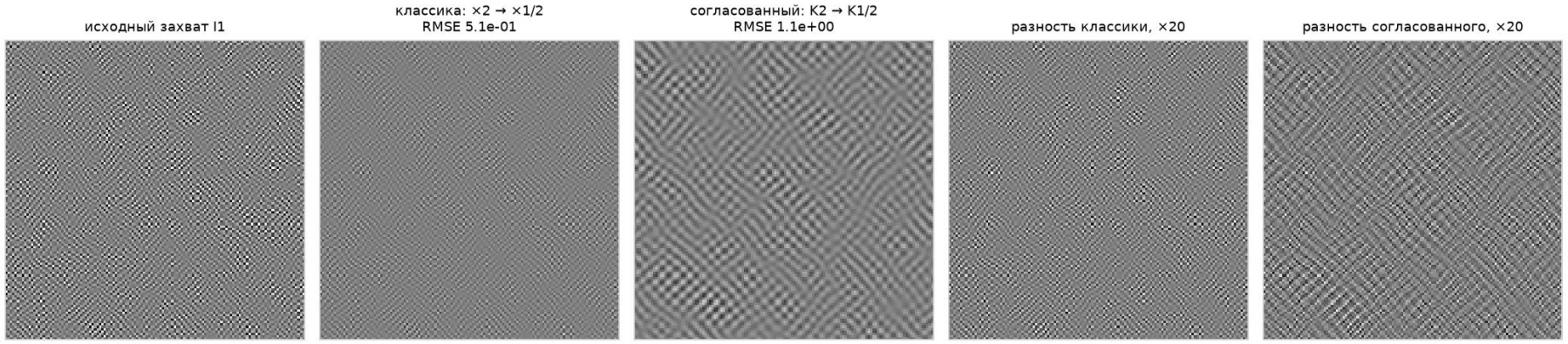
- Дано: кадр. Операции: ×2 затем ×1/2; два шага ×1/2 против одного ×1/4; тот же путь в обратном порядке
- Найти: ресайзер, для которого все три равенства точны. В библиотеках они не точны: RMSE $8.8 \cdot 10^{-3}$, $7.2 \cdot 10^{-4}$, $2.2 \cdot 10^{-3}$
- Согласованный оператор: 10^{-14}–10^{-16}, то есть машинная точность
- Цена ошибки: обучение SR наследует ваш даунсамплер; сопоставление после down-up теряет точки; классификатор меняет ответ от цепочки



© собственная иллюстрация

В чем главная проблема?

- Фундаментальная техническая проблема в компьютерном зрении (CV) : **стандартные библиотеки для изменения размера изображений** (например, в OpenCV, Pillow или PyTorch) **работают математически некорректно при последовательных операциях**, что незаметно портит результаты нейросетей;
- Популярные библиотеки (OpenCV, PIL и др.) используют стандартные алгоритмы интерполяции (билинейную, бикубическую). При их применении возникает погрешность (RMSE) на уровне 10^{-3}– 10^{-4}. Для человека это визуально незаметно, но для алгоритмов это огромная ошибка.



© собственная иллюстрация

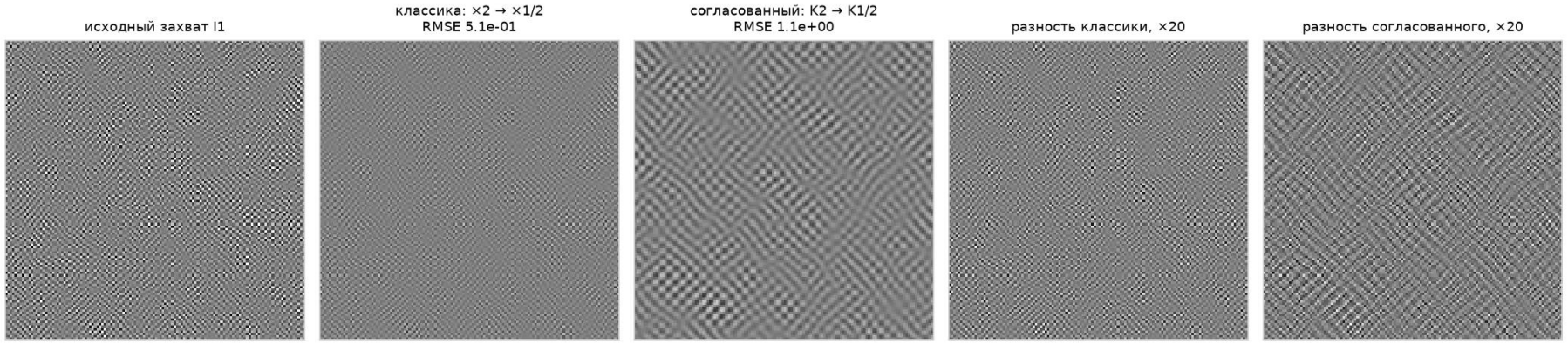
На что это влияет? («Цена ошибки»)

Кажется, что микроскопическая ошибка при изменении размера ни на что не влияет, но на слайде объясняется, почему это критично для разработчиков:

• **Обучение Super-Resolution (SR):** Если вы учите нейросеть восстанавливать качество картинок (делать апсемплинг), она «унаследует» и запомнит все математические дефекты кривого алгоритма сжатия (даунсамплера).

• **Потеря точек (Down-Up):** Если сжать картинку, а потом разжать обратно (down-up), то при сопоставлении пикселей или ключевых точек алгоритм начнет «терять» их координаты из-за сдвигов.

• **Сбой классификатора:** Обычная нейросеть-классификатор может выдать **совершенно другой ответ** на одну и ту же картинку просто из-за того, в каком порядке или какими промежуточными шагами её сжимали.



© собственная иллюстрация

На что это влияет? («Цена ошибки»)

arXiv:2610.22472 [eess.IV] | 15 Oct 2026

A Unified Theory of Image Scaling:

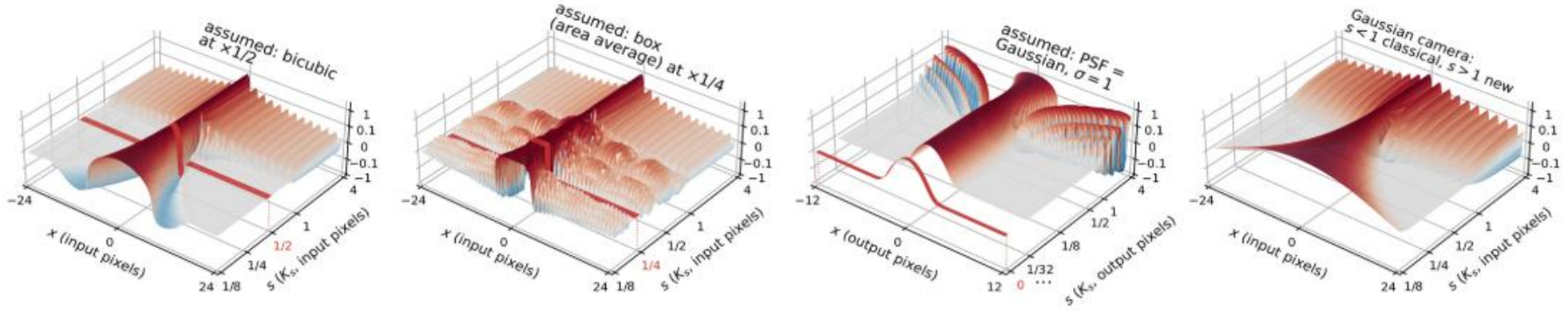
Point Spread Functions, Discrete Kernels, and Scale-Spaces

Assaf Shocher

Faculty of Data and Decision Sciences

Technion — Israel Institute of Technology

Paper | Code | Video | BibTeX



One specified kernel or point spread function determines a family of kernels across scales. The given slice is marked in red.

© Assaf Schocher, Oct 2026, https://assafshocher.github.io/image-scaling/

Почему точка становится пятном? (PSF)

Рассмотрим **как реальный мир проецируется на матрицу цифровой камеры**, и почему идеальная точка объекта всегда превращается в размытое пятно на снимке.

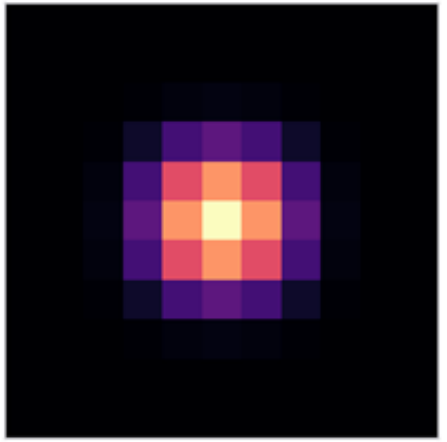
Рассмотрим фундаментальное понятие **PSF (Point Spread Function — функция рассеяния точки)** и объясним, почему старая мат. модель (размытие по Гауссу) десятилетиями всех устраивала, но глобально устарела.

1. Почему точка становится пятном? $\left(\boldsymbol{h = h_{lens} * h_{pix}}\right)$

В идеальном мире точка — это один бесконечно малый пиксель. В реальности, когда свет проходит через камеру, происходит два искажения:

- **Физика линзы (h_{lens}):** Из-за дифракции света и несовершенства стекла (аберраций) объектив слегка размывает луч.
- **Цифровая матрица (h_{pix}):** Пиксель на матрице имеет физический размер. Свет «интегрируется» (собирается) по всей площади этого квадратика.
- **Итог:** Комбинация этих факторов (математически — свёртка (*)) дает функцию (*h*) (PSF) - **одна яркая точка превращается в «крест» или пятно размером примерно в 1 пиксель ($\sigma \approx 1px$)**.

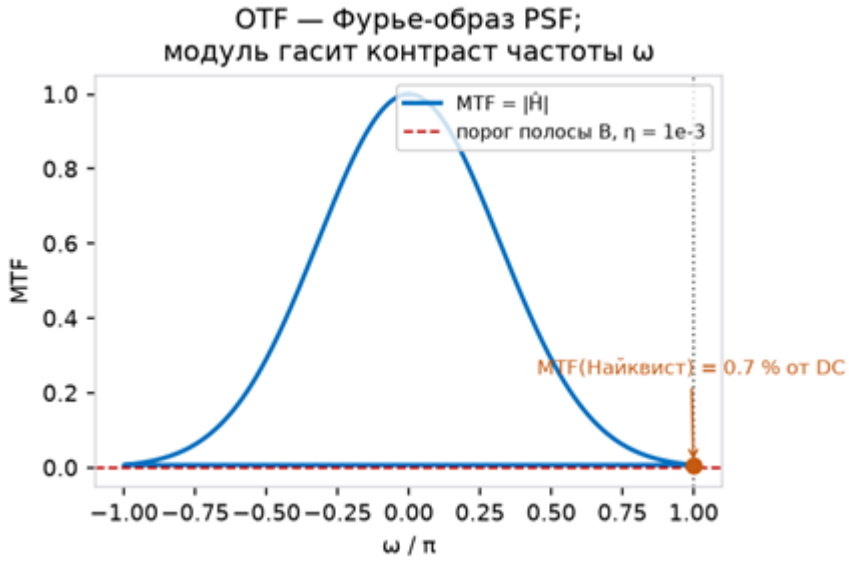
PSF h = h_lens ∗ h_pix:
дифракция/аберрации + пиксель, σ ≈ 1 px



Как оценить зоркость камеры? (OTF и MTF)

Чтобы понять, насколько хорошо камера передает мелкие детали, ученые переводят картинку из пространства пикселей в пространство частот с помощью преобразования Фурье:

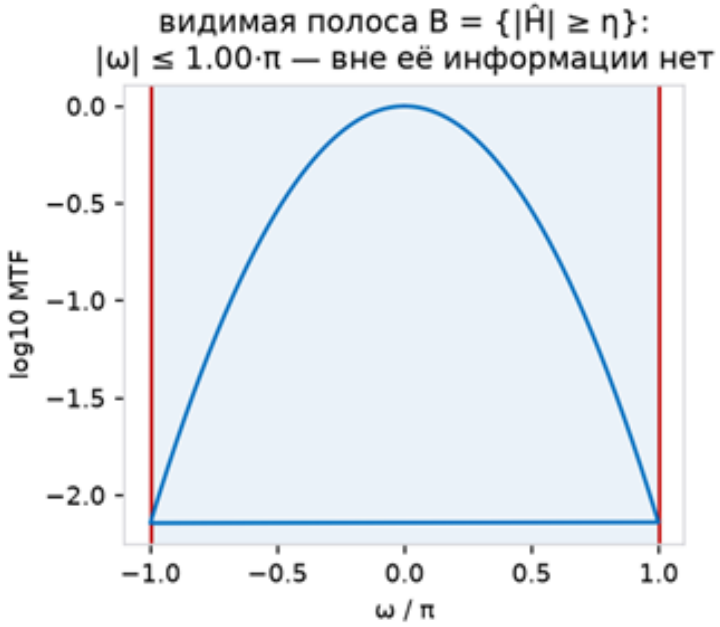
- **OTF (Optical Transfer Function):** Это «частотный паспорт» нашей оптической системы.
- **MTF (Modulation Transfer Function):** Модуль этой функции. Простыми словами — **график того, как сильно камера гасит контраст в зависимости от мелкоты деталей (ω)**.
  - **Низкие частоты (центр графика, 0.0):** Крупные объекты (например, белая стена). Контраст идеальный (1.0).
  - **Высокие частоты (края графика, 1.0):** Сверхмелкие детали (штрихи, текстуры). Камера их «мылит», контраст падает к нулю.



Феномен Гаусса: почему старая модель работала «десятилетиями»?

Часто функцию размытия камеры (*h*) моделировали обычной Гауссианой (размытием по Гауссу с радиусом $\boldsymbol{\sigma \approx 1px}$).

- Существует **критический предел четкости матрицы** — частота Найквиста (на графиках отмечена как $1.00 \cdot \pi$). Всё, что мельче этого предела, цифровая камера в принципе не способна зафиксировать (начинаются искажения).
- На этой критической частоте график MTF Гаусса падает практически до нуля — он равен всего 0.7% от исходного контраста (DC).
- Вывод: Поскольку реальная камера всё равно «слепнет» на этой частоте, размытие по Гауссу идеально имитировало поведение реальной оптики внутри видимого диапазона. На графике (в логарифмическом масштабе) четко видна «видимая полоса B», вне которой информации для камеры просто не существует.

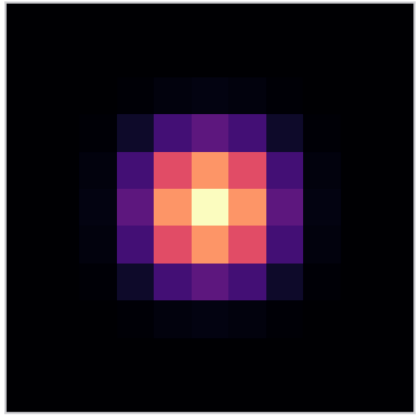


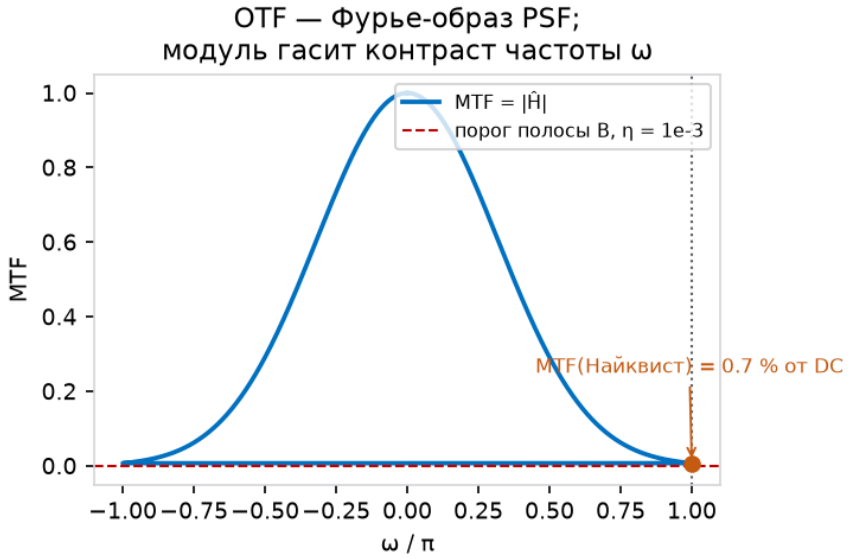
Когда библиотеки (вроде OpenCV) сжимают или увеличивают картинку, они делают это чисто математически, полностью игнорируя физику камеры (PSF, MTF и частоту Найквиста). Настоящий же, «согласованный» оператор ресайза должен учитывать эти физические ограничения сенсора, чтобы не искажать информацию, которую нейросеть пытается извлечь из пикселей.

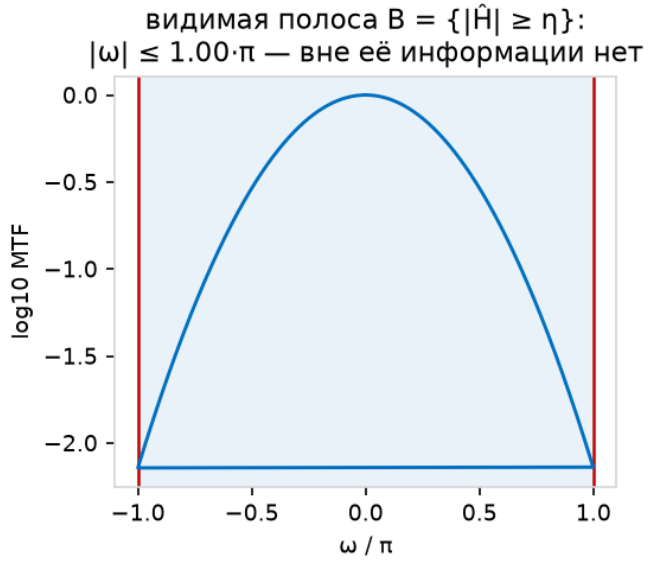
PSF (Point Spread Function): точка становится пятном

- `h = hlens ∗ hpix`: дифракция и аберрации линзы, сложенные с интегрированием по пикселю
- OTF — Фурье-образ h; MTF — модуль OTF; видна камера там, где MTF ≠ 0: это полоса B
- Гаусс σ = 1 px: MTF на Найквисте 0.7 % от DC — поэтому Гаусс «служил» моделью десятилетиями

PSF h = h_lens ∗ h_pix:
дифракция/аберрации + пиксель, σ ≈ 1 px







© собственная иллюстрация

Формула кадра: Что на самом деле видит сенсор?

Формула $\boldsymbol{I[n] = (I_{cont} * h)(n)}$ расшифровывается так:

- $\boldsymbol{I_{cont}}$ — непрерывный реальный мир (бесконечно четкая сцена).
- $\boldsymbol{h}$ — та самая функция размытия оптики и пикселя (PSF).
- $*$ (свёртка) — процесс «замыливания» мира оптикой.
- $[\boldsymbol{n}]$ — сетка пикселей (матрица), которая ловит этот результат в фиксированных точках.

Физический смысл: Сенсор принципиально не способен зафиксировать идеальную реальность. Он видит её сглаженную копию, из которой оптика уже «вырезала» (band-limited) все сверхмелкие детали. Всё, что было за пределами полосы пропускания оптики (полосы B), **потеряно навсегда еще до того, как превратилось в пиксели**.

Парадокс оптического зума: «У каждого масштаба своя камера»

Когда вы делаете оптический зум (изменяете масштаб сцены в *s* раз), физика процесса меняется:

- Реальный мир на матрице растягивается.
- Но физический размер пикселя на матрице остаётся прежним!
- **Итог:** Математическое ядро размытия *h* в пиксельных координатах драматически меняется по формуле $(s \cdot M_{\frac{1}{s}} h)$. С точки зрения математики, **каждое изменение фокусного расстояния (зум) — это как будто мы вставили в устройство совершенно другую матрицу с другой оптикой.**

Следствие: фиксированное ядро интерполяции не может совпадать с оптикой на всех масштабах. Когда стандартные библиотеки (OpenCV, PyTorch) делают цифровой ресайз (например, уменьшают картинку в 2 раза), они используют одно и то же фиксированное математическое правило (например, бикубическую интерполяцию).Но реальная камера при изменении масштаба меняет своё физическое ядро

Формирование изображения: одна формула

Дано: яркость сцены `Icont`. Кадр: `I[n] = (Icont ∗ h)(n)` — свёртка с PSF, взятая в узлах сетки

Что это значит физически: сенсор видит не сцену, а её сглаженную оптикой band-limited копию; вне полосы B информации нет

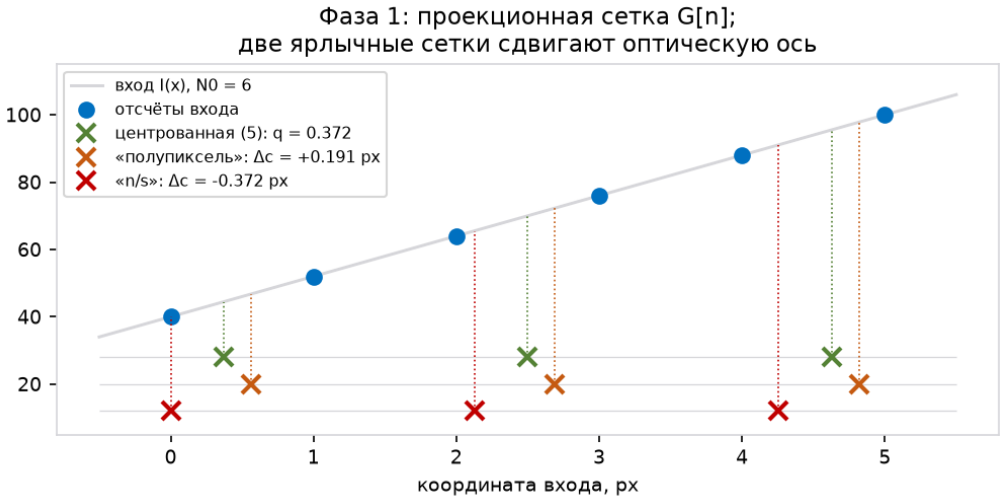
Оптический зум: растянуть сценю в `s` раз — ядро меняется на `(s·M1/s h)`: у каждого масштаба своя камера в пиксельных координатах

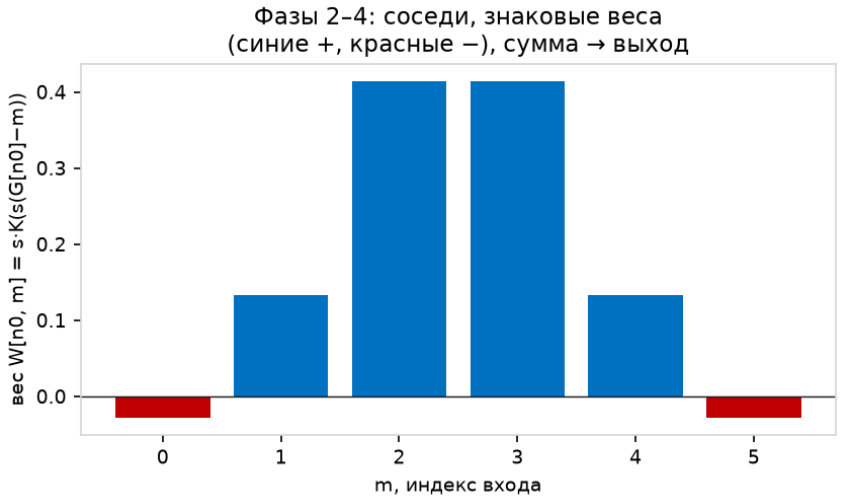
Следствие: фиксированное ядро интерполяции не может совпадать с оптикой на всех масштабах

Кадр — не сцена: из него восстанавливаемо только то, что лежит в полосе B. Цифровой снимок — это лишь ограниченный набор данных. Мы можем математически точно восстановить или обработать только ту информацию, которая смогла пробиться через линзу и уместилась в «полосу B». Любая попытка делать ресайз без учета физики этого фильтра (оптики конкретной камеры) приводит к накоплению математических ошибок, которые затем путают нейросети.

Четыре фазы любого resize

- N_0 — количество пикселей во **входном** изображении (на графиках $N_0 = 6$).
- N — количество пикселей в **выходном** (новом) изображении.
- $s = \frac{N}{N_0}$ — масштабный коэффициент (**scale**).
  - Если $s > 1$, изображение увеличивается (upscale).
  - Если $s < 1$, изображение уменьшается (downscale).
- n — индекс пикселя на *выходном* изображении.
- $G[n]$ — координата центра выходного пикселя n, спроецированная обратно на координатную сетку *входа*.





© собственная иллюстрация

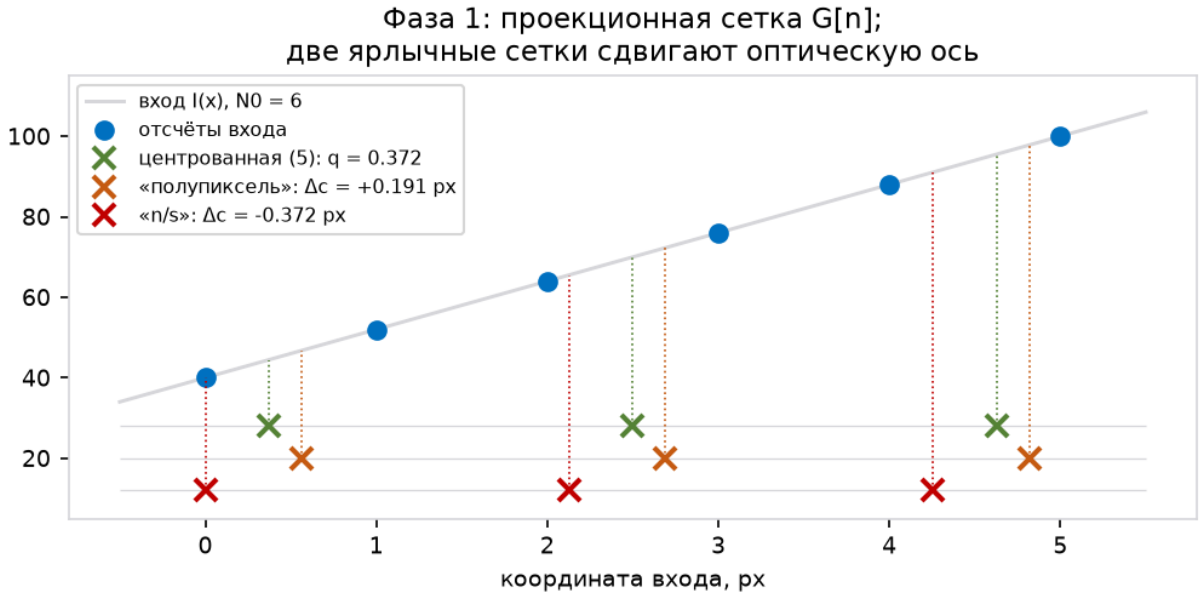
Фаза 1: Сетка (Проекция координат)

Когда мы создаем новое изображение, мы должны понять: *какой точке старого изображения соответствует центр каждого нового пикселя?*
Формула «идеального» центрирования сеток:

$$G[n] = \frac{n}{s} + \frac{N_0 - 1}{2} - \frac{N - 1}{2s}$$

Что формула делает?

- Член $\frac{n}{s}$ просто масштабирует шаг сетки.
- Остальная часть формулы гарантирует, что **центр исходного изображения совпадет с центром нового изображения**. Без этого сдвига картинка при изменении размера «уползет» в угол.



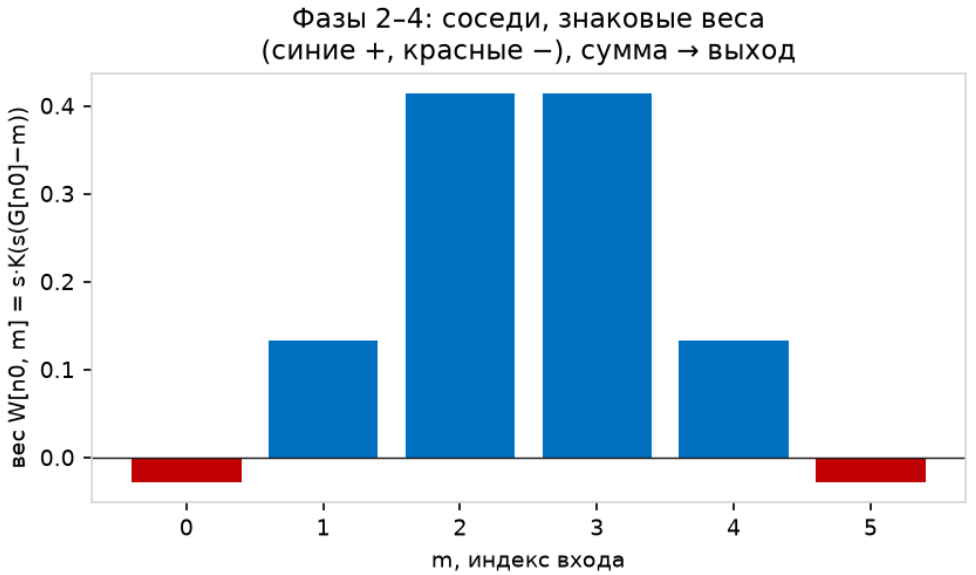
© собственная иллюстрация

Фазы 2–3: Сбор соседей и интерполяция

После того как мы нашли точку $G[n]$ на входной сетке (она почти никогда не попадает ровно в целый пиксель), нам нужно вычислить значение цвета в этой точке.

- **Сбор соседей:** Мы берем непрерывную функцию (интерполянт, например, бикубический сплайн или фильтр Ланцоша) и центрируем её в точке $G[n]$.

- **Знаковые веса (знаковый сдвиг):** Качественные фильтры интерполяции (такие как кубические сплайны или Lanczos) имеют «волны» — у них есть как положительные, так и отрицательные веса.
  - На графике видно: центральные пиксели *(m=2, 3)* имеют **положительные веса (синие столбики)**.
  - Пиксели на краях *(m=0, 5)* имеют отрицательные веса (красные столбики), которые нужны для создания эффекта «шарпинга» (резкости) на границах объектов, чтобы изображение не выглядело размытым после ресайза.



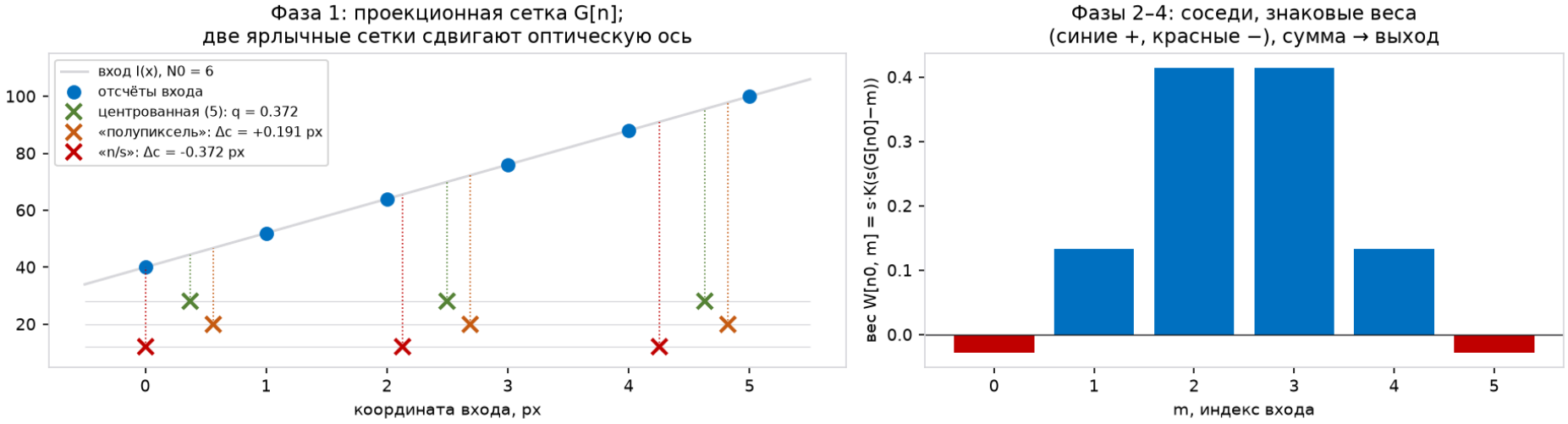
© собственная иллюстрация

Фаза 4: Сумма и Антиалиасинг (Downscale)

Когда изображение уменьшается (s < 1), возникает проблема **алиасинга (муара/подмены частот)**. Если мы возьмем слишком узкое ядро интерполяции, мы начнем пропускать пиксели, и на картинке появятся «зубцы» или ложные узоры.

• Решение: На фазе 4 ядро фильтра нужно растянуть и нормировать (сделать шире), чтобы оно физически усредняло большую область входных пикселей.

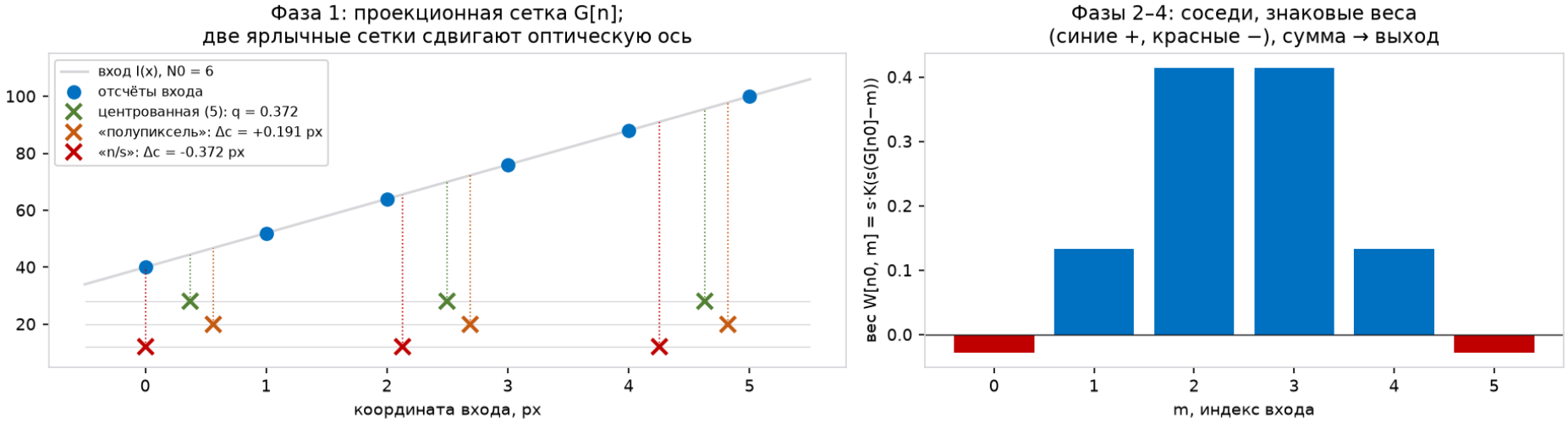
• Если этого не сделать («иначе подмена частот»), мелкие детали превратятся в кашу и артефакты.



© собственная иллюстрация

Ловушка сеток: Ошибка округления в библиотеках

- В старых или простых библиотеках (и даже в некоторых ранних версиях OpenCV/PIL) использовались упрощенные формулы проекции координат. Например, простое правило $\frac{n}{s}$ или классическое «полупиксельное» выравнивание.
- **К чему это ведет на практике?** Если вы делаете resize картинки много раз подряд по плохим формулам (например, в глубоких нейросетях при bounding box конструировании), картинка с каждым шагом будет незаметно сдвигаться в сторону на доли пикселя, ломая точность детекции или сегментации.



© собственная иллюстрация

Несогласованность практики: три провала

Стандартные математические алгоритмы изменения размера (resize) в современных библиотеках фундаментально некорректны, поскольку они не учитывают физику реальной камеры.

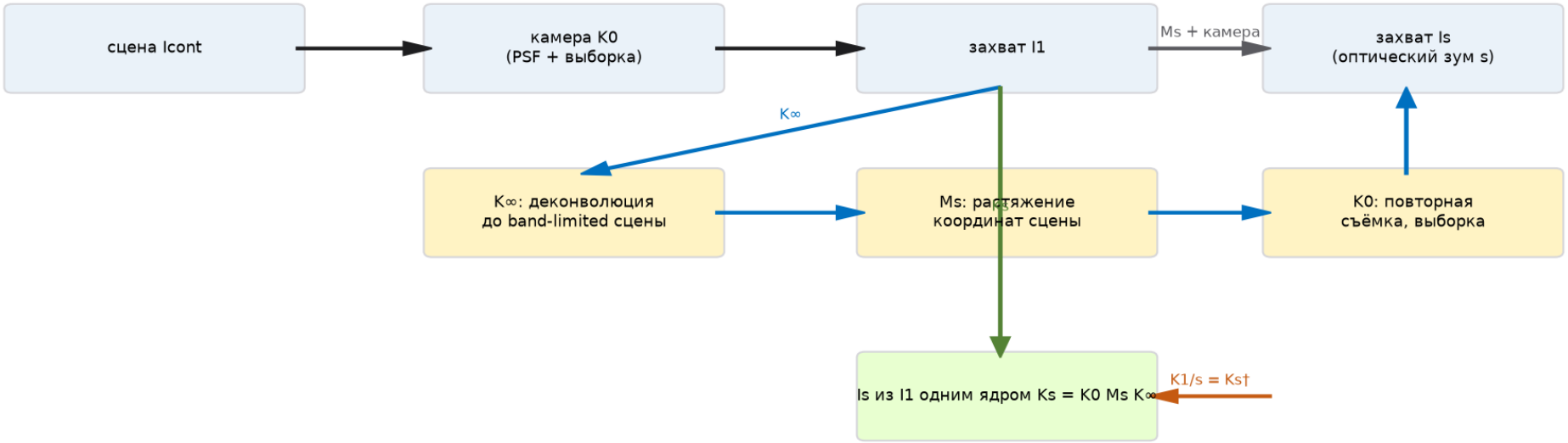
Из-за этого нарушаются базовые алгебраические свойства, что приводит к трем практическим провалам (ошибкам RMSE):

- **Провал цикла (× 2 затем × 1/2 ≠ тождество):**
- Если увеличить картинку в два раза, а потом уменьшить обратно, мы не получим исходное изображение. Ошибка огромна, особенно на реальной оптике («камера-диск»).
- **Провал композиции (× 1/1.6 затем × 1/2.5 ≠ × 1/4):** Последовательный ресайз в несколько шагов дает другой результат, нежели однократное сжатие до того же итогового размера.
- **Провал коммутативности:** Изменение порядка двух последовательных уменьшений (например, сначала по ширине, потом по высоте) меняет финальный ответ.

Это не случайные баги или ошибки программистов в OpenCV или PIL. Это фундаментальная проблема теории: правила ресайза создавались в отрыве от физических свойств сенсоров и оптики камер.

Пиксели как вторичные источники

- Идея Гюйгенса для волн: каждая точка фронта — источник; здесь каждый пиксель «излучает» сигнал
- Суперпозиция пикселей восстанавливает непрерывную band-limited сцену из кадра
- Растянуть её и снять той же камерой — это и есть согласованный зум, без обращения к исходной сцене

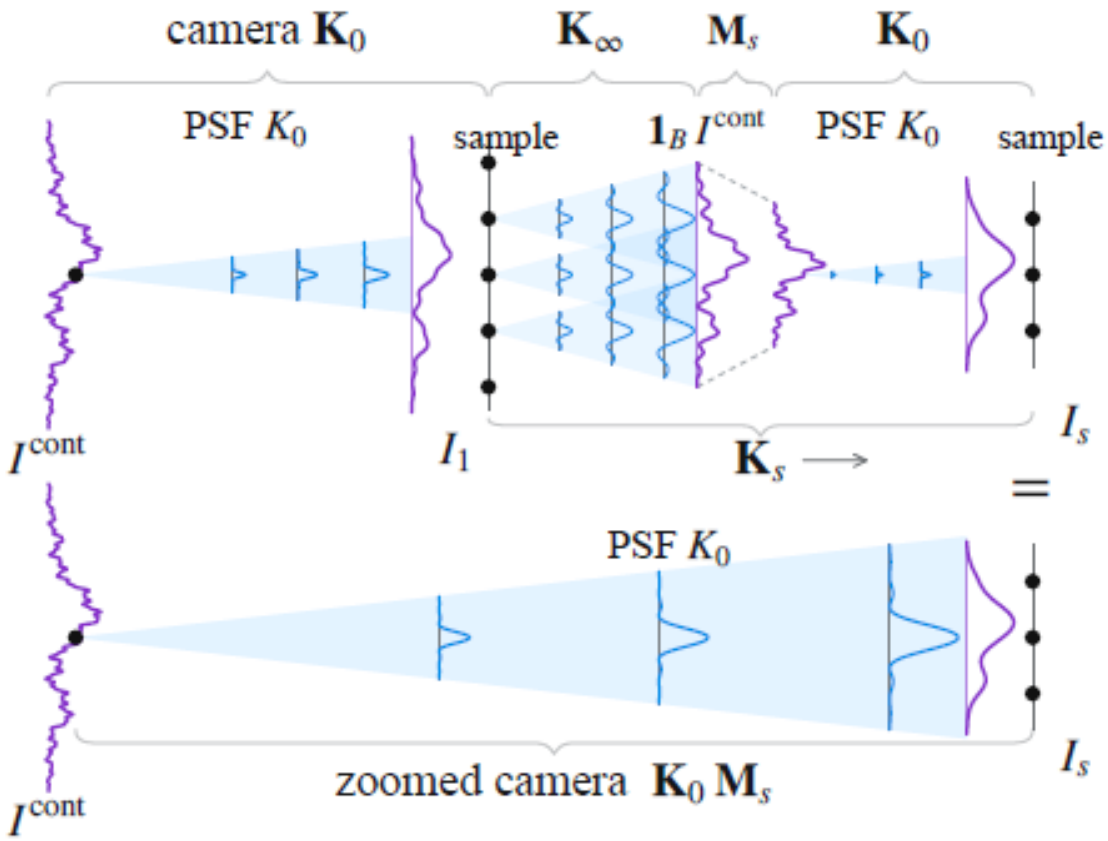


Пиксели — вторичные источники: суперпозиция восстанавливает band-limited сцену;
растянуть и снять той же камерой = оптический зум; обратный путь возвращает разрешение, не частоты

© собственная иллюстрация по мотивам Fig. 1 статьи (Shocher, 2026)

Двунаправленное масштабирование по принципу Гюйгенса

- Первый этап (camera K_0): Непрерывная сцена I^{cont} проходит сквозь оптику камеры с размытием ядра оптики (PSF K_0) и дискретизируется матрицей (sample), превращаясь в исходный цифровой кадр I_1.
- Второй этап (camera K_∞): Каждый дискретный пиксель кадра I_1 начинает «излучать» (как вторичный источник по Гюйгенсу), восстанавливая непрерывный частотно-ограниченный сигнал $1_B I^{cont}$.
- Третий этап (M_s): Восстановленная непрерывная сцена пространственно растягивается в *s* раз.
- Четвертый этап (K_0): Растянутый сигнал виртуально «снимается» заново: он снова проходит через то же самое физическое размытие оптики (PSF K_0) и субпиксельную выборку (**sample**), давая идеальный цифровой зум-кадр I_S.



© собственная иллюстрация по мотивам Fig. 1 статьи (Shocher, 2026)

Мастер-формула: $K_s = K_0 \circ M_s \circ K_\infty$

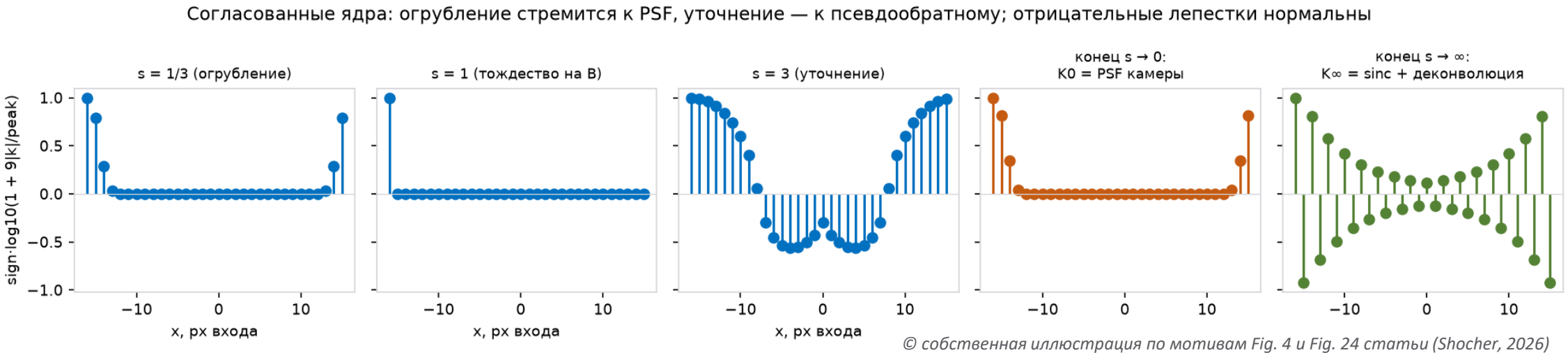
- K = $K_0^\dagger$ — псевдообратный по Муре–Пенроузу: деконволюция до непрерывной band-limited сцены
- M_s —растяжение координат сцены в s раз; K_0 —повторная съёмка той же камерой, с выборкой
- В частотах ядро просто: $\widehat{K}_s(\omega) = \widehat{K}_0(\omega s) \;/\; \widehat{K}_0(\omega)$ — отношение передаточных функций камеры
- При s > 1 новых частот нет: каноническое дополнение — минимум нормы, псевдообратный даунсамплера

Одна камера фиксирует ядра на всех масштабах и фазах сетки; свобода остаётся лишь вне полосы B

Суть формулы: Она показывает, как соотносятся между собой исходный снимок, оптическое размытие и цифровое изменение масштаба (зум).

Семейство ядер K_s и его три конца

- $\lim_{s\to 0} K_s = K_0$: бесконечно плотная выборка (сверхвысокое разрешение). Процесс изменения масштаба (ресайз) превращается в само формирование кадра оптической системой.
- K1 = Id на достижимых кадрах: при s = 1 ядро — band-limited дельта-функция, а не идеальный Дирак, так как полоса частот ограничена
- $\lim_{s\to\infty} K_s = K_\infty$: предельное уточнение — sinc-интерполяция, совмещенная с деконволюцией функции рассеяния точки (PSF) объектива.
- Телескопическое произведение: один измеренный зум-керн (ядро масштабирования) позволяет восстановить свойства и параметры камеры целиком.



© собственная иллюстрация по мотивам Fig. 4 и Fig. 24 статьи (Shocher, 2026)

Свойства семьи и обобщённый scale-space

Алгебраические свойства:

- Композиция: $Ks' \circ Ks = Kss'$, кроме пути «вниз-затем-вверх»
- Коммутативность в одну сторону:

Порядок выполнения масштабирования в одинаковых направлениях не имеет значения.

- Обращение: $K1/s = Ks^{\dagger}$; ядро Ks единственно

Сравнение с классическим Scale-space:

- В классической теории (полугруппа, причинность, подобие…) единственным решением является **Гауссиан** (размытие Гауссом).
- **Новшества автора:** Вводятся добавки (точность на полосе *B* и двунаправленность).
- Отмечается, что Гаусс не может быть реальной PSF конечной апертуры, так как его оптическая передаточная функция (OTF) не имеет строгого обрезания по частоте.
- Уточнение $s > 1: t(s) < 0$ — это деблюринг, а не сглаживание, как в классическом Гауссовом scale-space

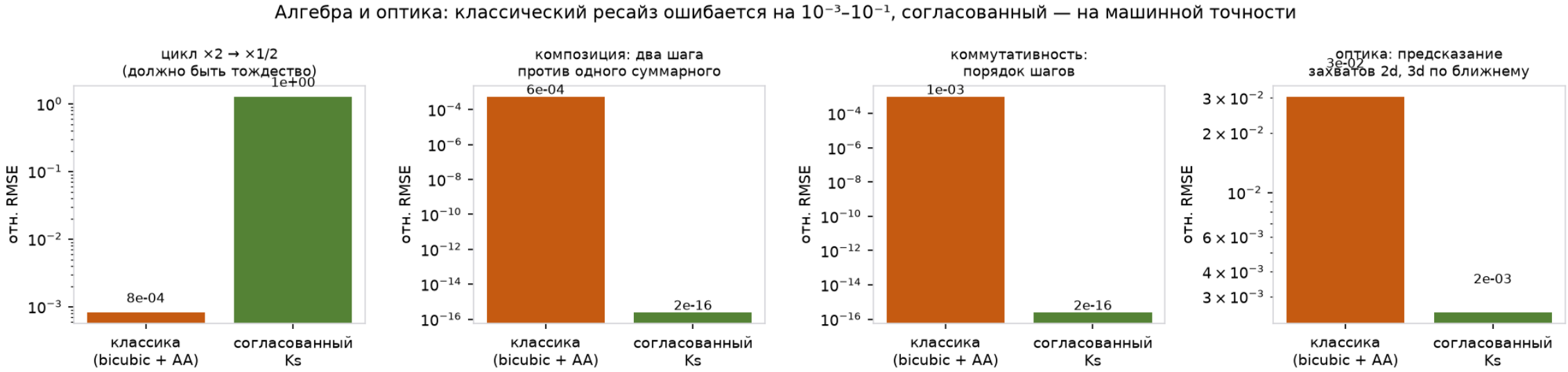
3

Проверка, приложения, границы

Числа тестов · где согласованность платит децибелами · практический рецепт · пределы теории

Тесты: цикл, композиция, поворот

- Цикл ×2 → ×1/2: классика $8.8 \cdot 10^{-3}$ и $3.1 \cdot 10^{-2}$, согласованный 10^{-14}–10^{-16}
- Композиция ×1/1.6→×1/2.5 против ×1/4: $7.2 \cdot 10^{-4}$ против $2 \cdot 10^{-15}$; коммутативность: $2.2 \cdot 10^{-3}$ против $4 \cdot 10^{-15}$
- Поворот 15° + сдвиг (0.37, 0.21) px и обратно: $4.2 \cdot 10^{-3}$ против $4 \cdot 10^{-16}$
- Оптика: предсказание захватов 2d, 3d по ближнему — rel RMSE 0.15–0.20 против 0.53–1.05 у cubic AA



© собственная иллюстрация по мотивам Fig. 6–8 статьи (Shocher, 2026)

Где согласованность платит

Задача	Метрика	Согласованный против классики
Апскейл с известным blur, BSD100 ×2	Y-PSNR, дБ	30.65 против 25.88 (Lanczos) и 25.87 (Фурье)
Супер-разрешение ZSSR, 10 шагов	прирост к 1 шагу	+2.58…+5.02 дБ от закона масштаба обучения
Сопоставление SIFT после down-up, HPatches 32×	успех, п.п.	+17.1 / +9.1 / +5.9 против linear / cubic / Lanczos AA
Устойчивость классификатора к цепочке ресайзов	смена top-1, %	16–49 % у библиотек против ≈ 0 % у семьи камеры
Перенос ядра ×2 → ×3, ×4, RealSR	медианный PSNR	31.5 против 31.3 (AA stretch) при фит-эталоне 32.9

На синтетике выигрыши огромны, на реальных парах — скромны: регистрация и шум съедают половину

Практика: когда консистентность нужна

- Нужна: цепочки масштабов — обучение SR, аугментации, пирамиды сопоставления, down-up matching, предсказание захвата
- Достаточно обычного AA: одиночный ресайз для показа или обучения без цепочки; разница тонет в шуме данных
- Требуется на входе: калиброванный PSF (или оценка из зум-пары телескопией), сдвиг-инвариантность, alias-free выборка
- Границы: за Найквистом сенсора алгебра остаётся, оптика — нет; оптика меняется с фокусом и зумом; квантование не моделируется

Согласованный ресайз — не новый интерполятор, а проекция на сохранённую полосу частот

Типичные ошибки масштабирования

ПОДВОДНЫЕ КАМНИ

- Смешать масштаб и размер: s задаёт шаг, N — охват; круговой путь требует обоих, иначе зум уезжает на $9/5$ вместо 2
- Сетка-ярлык: правило n/s центрирует первый пиксель, не ось; при $N \neq sN_0$ ось уезжает на $\Delta c = (N - sN_0)/(2s)$
- Гауссов AA с приращением $s^{-2}\kappa^2$ вместо $(s^{-2} - 1)\kappa^2$: это не оптическая камера ни при каком σ (Remark 5.2)
- Ждать деталей от down-up: цикл возвращает проекцию на достижимое подпространство, а не исходник; новые частоты неоткуда

Что унести с собой

KEY TAKEAWAYS

- Кадр — это $(I_{cont} * h)(n)$: восстанавливаемо лишь содержимое полосы B; PSF = линза $*$ пиксель, её Фурье-образ задаёт MTF
- Resize — четыре фазы; сетка $G[n] = n/s + (N_0-1)/2 - (N-1)/(2s)$; ярлыки сеток сдвигают оптическую ось
- Практика несогласованна: цикл $8.8\cdot10^{-3}$, композиция $7.2\cdot10^{-4}$; чистое AA-растяжение не соответствует ни одной камере (Cor 3.4)
- Согласованный оператор $K_s = K_0\ M_s\ K_\infty$ композируется, коммутирует, обращается как $K_{1/s} = K_s^\dagger$; концы: K_0, Id, K_∞
- Платит там, где цепочка масштабов: SR +2.6–5 дБ, HPatches 32× +17 п.п., флапы классификатора → 0; одиночный ресайз — хватит AA

Вопросы для самопроверки

САМОПРОВЕРКА

- Почему Гаусс не может быть точным PSF реальной камеры, и что именно ломается в его OTF?
- Кадр уменьшили вчетверо и вернули ×4. Что вернётся, а что нет — и почему это проекция, а не интерполяция?
- Ядро $\hat{K_s}(\omega) = K_0(\omega s)/K_0(\omega)$: чему оно равно при s = 1 и к чему стремится при s → 0?
- В каком случае согласованный ресайз и «matched AA» дадут один ответ на ×1/2 и разойдутся на ×1/3?

Разбор — в начале следующей лекции, на слайде «Опоры»

Занятие 10 и мосты

ДАЛЬШЕ

- Занятие 10: аудит ресайзера по четырём фазам; тесты цикла/композиции/коммутативности на numpy; оценка ядра камеры по зум-паре (телескопия)
- ⭐ Согласованный down-up для сопоставления точек: сравнить с `INTER_AREA` на паре кадров ДЗ 3
- Мосты: пирамиды L6 получают согласованную сетку; гомография и трекинг (L10-homography, L13) — сопоставление при разных масштабах
- Данные и ноутбук — в репозитории за сутки до пары; PSF-калибровка стенда лежит в `demo/psf-calib.npz`

References

REFERENCES

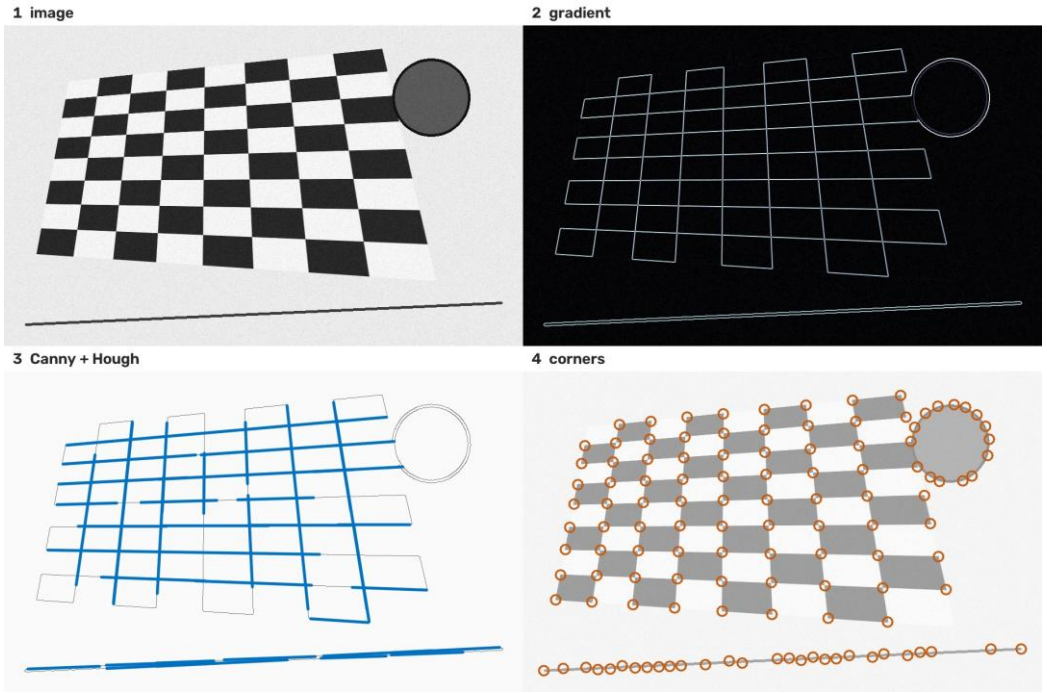
- Shocher A. *A Unified Theory of Image Scaling: Point Spread Functions, Discrete Kernels, and Scale-Spaces*. arXiv:2610.22472, 2026 — разделы 1–5 и App. C–D
- Lindeberg T. *Scale-Space Theory in Computer Vision*. Kluwer, 1994; Koenderink J. *The structure of images*. Biological Cybernetics, 1984
- Unser M., Aldroubi A. *A General Sampling Theory for Nonideal Acquisition Devices*. IEEE Trans. Signal Processing, 1994
- Michaeli T., Irani M. *Nonparametric Blind Super-Resolution*. ICCV 2013 — отношение передаточных функций между масштабами
- Burt P., Adelson E. *The Laplacian Pyramid as a Compact Image Code*. IEEE Trans. Communications, 1983
- Keys R. *Cubic Convolution Interpolation*. IEEE TASSP, 1981; Duchon C. *Lanczos Filtering*. J. Applied Meteorology, 1979

Толковый словарь лекции

ТОЛКОВЫЙ СЛОВАРЬ

Термин	Что это значит
PSF (point spread function)	отклик камеры на точку: пятно на сенсоре; свёртка линзы и интегрирования пикселем
OTF / MTF	Фурье-образ PSF / его модуль: во сколько раз камера передаёт контраст каждой частоты
видимая полоса B	частоты с ненулевым MTF; только они присутствуют в кадре и восстанавливаемы
band-limited сигнал	сигнал со спектром, ограниченным полосой: кадр после оптики всегда такой
дискретное ядро масштабирования	фильтр k, свёртка с которым предшествует прореживанию: $I_s = (I * k)\downarrow$
проекционная сетка	центры выходных пикселей в координатах входа; фиксирует оптическую ось и зум
антиалиасинг (AA)	подавление частот выше нового Найквиста до прореживания; иначе подмена
псевдообратный (Moore–Penrose)	обращение на достижимых выходах и дополнение минимальной нормы
деконволюция K^{∞}	подъём кадра до непрерывной band-limited сцены псевдообратным PSF
согласованный оператор K_s	ресайз, воспроизводящий оптический зум той же камеры: композируется и обращается
scale-space	семейство всё более сглаженных кадров; классические аксиомы дают Гаусс
обобщённый scale-space	семейство K_s одной камеры: точность на полосе B и двунаправленность (B1–B2)

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru

Демо-код лекции: `lectures/L10-scaling/demo/`; статьи — arXiv:2610.22472